EGC 211P:
Introduction to Object Oriented Design *using C++*

T1-26-27
Ahana Pradhan

Course Objectives

- Introduction to software design
- Understanding of object oriented concepts
- Build proficiency in **C++**
- Get used to IDE-based (not LLM-based) debugging and development

Course Structure

Class Schedule

- 2 lectures and 1 lab each week

Grade breakup –

- In-class quizzes (2) – 10% total
  - 2 announced quizzes – pen-paper – subjective conceptual questions (15-20 mins each)
- Attendance + class participation + LMS quiz during class hours – 10% (following the 80% rule)
- Lab assignment-based Viva (during lab hours) and take-home LMS quizzes – 30% total
- In-lab programming test (1) – 10% total (after GIE week)
- Final Exam (during Mid-sem) – 40% total

Assignment and Viva

1. Weekly programming assignment will be published.
2. **Lab submission to Domjudge within lab hours** (upto whatever is coded) **is mandatory for attendance**. Submission of the complete solution is ***optional, but is advised for building language skill***.
3. MCQ online quiz (take-home/during class hours) will be set on LMS. Submission will be counted for class-participation.
4. **During lab hours, viva will be conducted by choosing the candidates randomly** --> Instructor/TAs will roam around and will ask questions based on assignments done/submitted so far. If nothing is submitted, then will ask to **code in-place**.

Cheating / Plagiarism / Proxy / GenAI

- Do not do it! --> it will cause penalty in grades.
- Eventually lack-of-skill will show up in lab-viva

Reference Text Books:

Object-Oriented Analysis and Design with Applications: Grady Booch et al. https://zjnu2017.github.io/OOAD/reading/Object.Oriented.Analysis.and.Design.with.Applications.3rd.Edition.by.Booch.pdf

C++ Primer by Stanley Lippman et al.
https://zhjwpku.com/assets/pdf/books/C++.Primer.5th.Edition_2013.pdf

Office Hours

- Instructor
  - Available on most days in Room A210
  - Email in advance, if possible
- TAs for each group of students
  - TAs will manage their own communication with student groups

Bring Laptop to Lecture Classes

Type the code snippets shown in the slides (as instructed)

Anyone not having laptop, need to write code using pen-paper

Basics

```c
#include <stdio.h>

int main() {
    int age = 20;

    printf("Age = %d\n", age);

    return 0;
}
```

Basics

```c
#include <stdio.h>

struct Person {
    int age;
};

int main() {
    struct Person p;

    p.age = 20;

    printf("Age = %d\n", p.age);

    return 0;
}
```

Basics

```c
#include <stdio.h>

struct Person {
    int age;
};

typedef struct Person Person;

void birthday(Person* p) {
    p->age++;
}

int main() {
    Person p;
    p.age = 20;
    birthday(&p);
    printf("Age = %d\n", p.age);
    return 0;
}
```

C++

```cpp
#include <iostream>
using namespace std;

struct Person {
	int age;
	void birthday() {
		age++;
	}
};

int main() {
	Person p;
	p.age = 20;
	p.birthday();
	cout << "Age = " << p.age << endl;
}
```

Basics

```cpp
#include <iostream>
using namespace std;

struct Person {
	int age;
	void birthday() {
		age++;
	}
};

int main() {
	Person p;
	p.age = 20;
	p.birthday();
	cout << "Age = " << p.age << endl;
}
```

```cpp
#include <iostream>
using namespace std;

class Person {
public:
    int age;
    void birthday() {
        age++;
    }
};

int main() {
    Person p;
    p.age = 20;
    p.birthday();
    cout << "Age = " << p.age << endl;
}
```

Procedural vs OO Design

Let us Build a Story



Google Images

Tom is running at speed 5

Jerry is running at speed 7

Tom is chasing Jerry

..

NOUN VERB

Let Us **C**

```c
#include <stdio.h>
#include <string.h>

// --- Character Struct ---
struct Character {
    char name[20];
    int speed;
};
typedef struct Character Character;

// Function to make character run
void run(Character *c) {
    printf("%s is running at speed %d\n", c->name, c->speed);
}

// Function to simulate chase
void chase(Character *chaser, Character *target) {
    printf("%s is chasing %s\n", chaser->name, target->name);
    if (chaser->speed > target->speed)
        printf("%s caught %s!\n", chaser->name, target->name);
    else
        printf("%s escaped!\n", target->name);
}
```

Let Us **C**

```c
#include <stdio.h>
#include <string.h>

// 
stru

};
type

// 
void

}

// 
void

    printf("%s escaped!\n", target->name);
}
```

```c
int main() {
    Character tom = {.name: "Tom", .speed: 5};
    Character jerry = {.name: "Jerry", .speed: 7};

    run(&tom);
    run(&jerry);

    chase(chaser: &tom, target: &jerry);

    chase(chaser: &jerry, target: &tom);

    return 0;
}
```

Let us Build a Story



Google Images

Tom is running at speed 5

Jerry is running at speed 7

Tom is chasing Jerry

Tom places a MouseTrap for Jerry

A MouseTrap snaps near Jerry!

Tom places a BananaPeel for Jerry

..

NOUN VERB

Let Us **C**

```c
// --- Trap struct using function pointer ---
struct Trap {
    char name[20];
    void (*activate)(struct Trap*, Character*);
};
typedef struct Trap Trap;

// mouse trap
void activateMouseTrap(Trap *trap, Character *target) {
    printf("A %s snaps near %s!\n", trap->name, target->name);
    if (target->speed < 6)
        printf("%s gets caught by the trap!\n", target->name);
    else
        printf("%s dodges the trap!\n", target->name);
}

// banana peel
void activateBananaPeel(Trap *trap, Character *target) {
    printf("%s slips on a %s!\n", target->name, trap->name);
    printf("%s's speed is reduced!\n", target->name);
    target->speed -= 1;
}

// chaser places trap for target before chasing
void placeTrap(Character* chaser, Trap *trap, Character *target) {
    printf("%s places a %s for %s...\n", chaser->name, trap->name, target->name);
    trap->activate(trap, target);
}
```

Let Us **C**

```
int main() {
    Character tom = { .name: "Tom", .speed: 5};
    Character jerry = { .name: "Jerry", .speed: 7};

    run(&tom);
    run(&jerry);

    chase( chaser: &tom, target: &jerry);

    Trap mouseTrap = { .name: "MouseTrap", activateMouseTrap};
    Trap bananaPeel = { .name: "BananaPeel", activateBananaPeel};

    placeTrap( chaser: &tom, &mouseTrap, target: &jerry);
    placeTrap( chaser: &tom, &bananaPeel, target: &jerry);

    chase( chaser: &tom, target: &jerry);

    return 0;
}
```

Let Us **C**

What are the problems with the code?

- Focus on how to do, rather than who is doing
- Entities do not get importance in the design
- Difficult to model association between action and its actor
- Difficult to model the real-world

A Better Way of Modeling?

- Focus on who is doing what
  - (how is also specified, but we can abstract it when required)
- Actors are the first class citizen
- Strong association between actors and their actions
- Intuitive modeling of the real world

C

```c
int main() {
    Character tom = { .name: "Tom", .speed: 5};
    Character jerry = { .name: "Jerry", .speed: 7};

    run(&tom);
    run(&jerry);

    chase( chaser: &tom, target: &jerry);

    Trap mouseTrap = { .name: "MouseTrap", activateMouseTrap};
    Trap bananaPeel = { .name: "BananaPeel", activateBananaPeel};

    placeTrap( chaser: &tom, &mouseTrap, target: &jerry);
    placeTrap( chaser: &tom, &bananaPeel, target: &jerry);

    chase( chaser: &tom, target: &jerry);

    return 0;
}
```

C++

```cpp
int main() {
    Cat tom( s: 5);
    Mouse jerry( s: 7);

    tom.doRun();
    jerry.doRun();
    tom.doChase(jerry);

    // Create traps
    MouseTrap mt;
    BananaPeel bp;

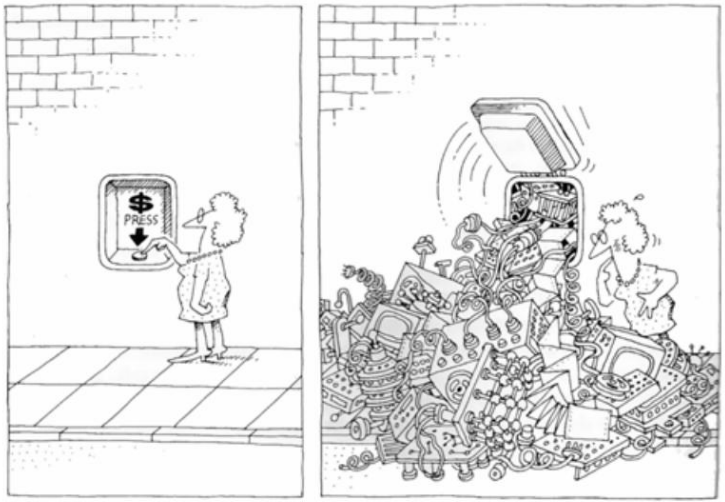
    tom.addTrap(&mt);
    tom.addTrap(&bp);

    // Tom deploys all his traps on Jerry
    tom.deployTraps( [&] jerry);
    tom.doChase(jerry);

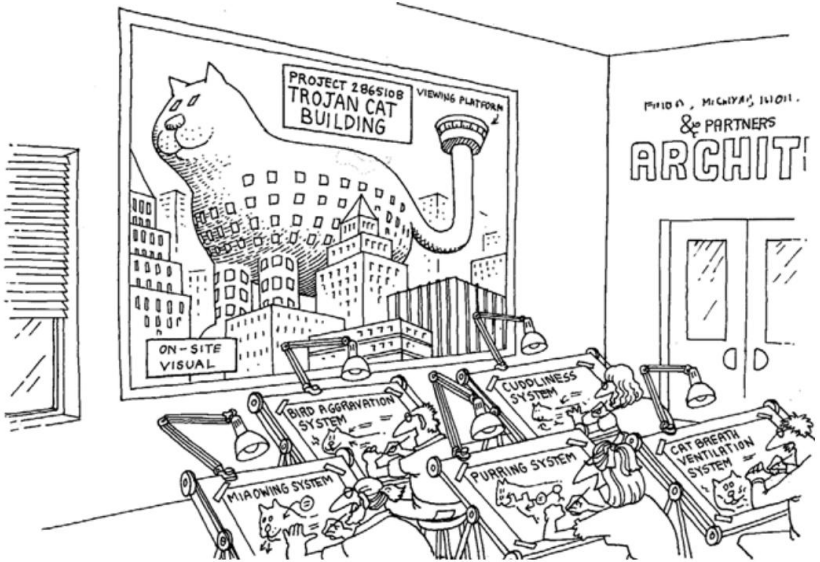
    return 0;
}
//
```

OO Design Key Ideas

OO Design Concepts



The task of the software development team is to engineer the illusion of simplicity.



The architecture of a complex system is a function of its components as well as the hierarchic relationships among these components.

Image source: Object-Oriented Analysis and Design with Applications by Grady Booch et al.

Objects and Classes

- Objects model elements (nouns) of the problem context
- *Class is like a Cookie-cutter. Objects are like the cookies.*



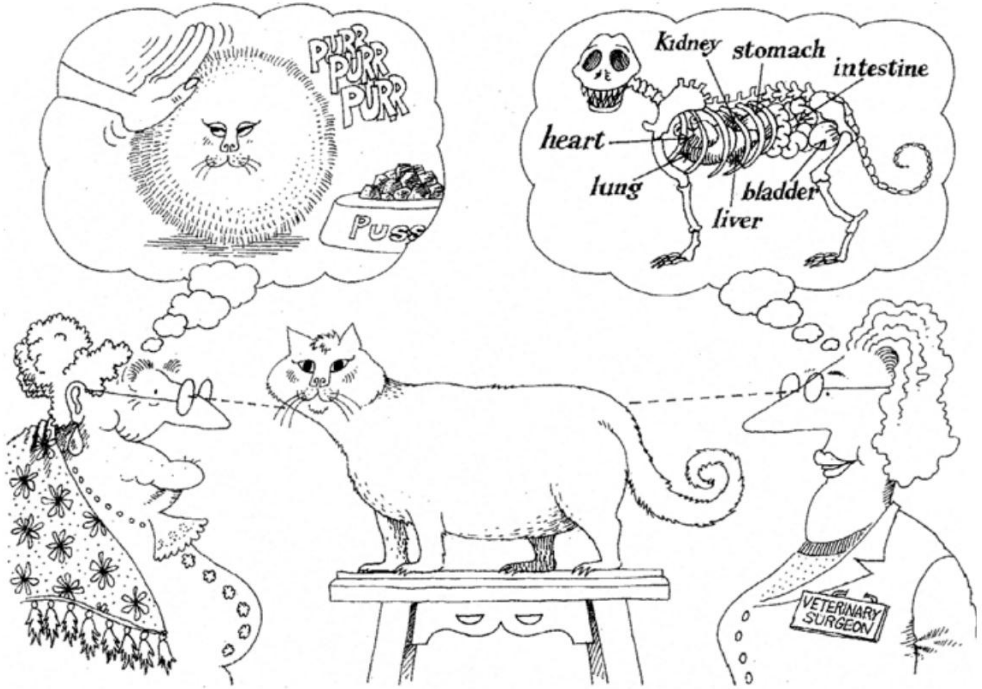
Google Images

- Their Characteristics: *Fields or attributes*
- Their Actions/Responsibilities/Promises (verbs): *Methods*

OO Design Concepts

- Abstraction
  - Represents a subject in the real world / domain
  - Has a level (higher level means less detail)
  - Identifies similarities and differences with other subjects
  - Identifies intrinsic characteristics of the subject
- Encapsulation
  - Packages a representation of a subject
  - Facilitates abstraction by hiding information:
    - A specification describes what an entity is and what it does
    - An implementation describes how an entity is realized
  - May contain data as well as procedures as elements

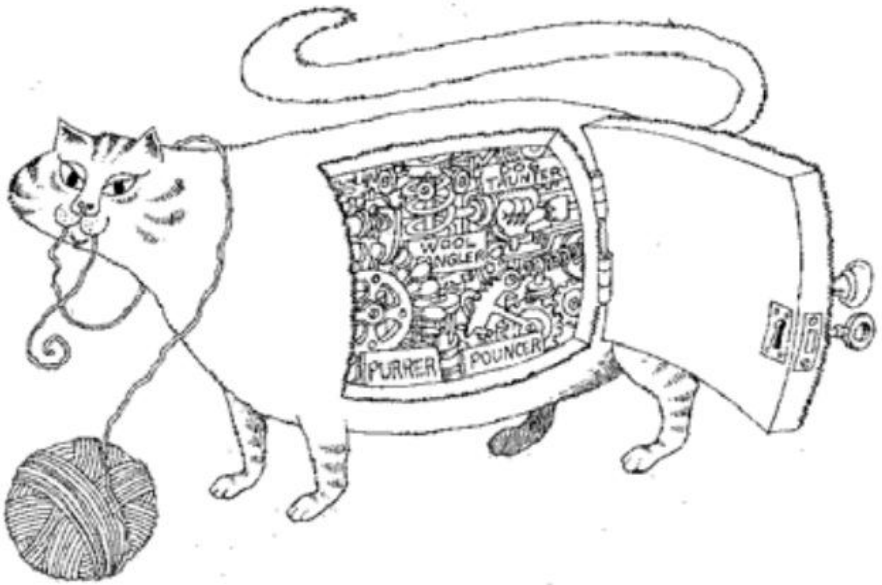
Abstraction



Abstraction focuses on the essential characteristics of some object, relative to the perspective of the viewer.

Image source: Object-Oriented Analysis and Design with Applications by Grady Booch et al.

Encapsulation



Encapsulation hides the details of the implementation of an object.

Image source: Object-Oriented Analysis and Design with Applications by Grady Booch et al.

OO Design Concepts

- Inheritance
  - Inheritance is a property derived from specialization, e.g. cat “is a kind of” animal so it “inherits” all the characteristics of an animal
  - A specialized (sub or derived) class can add its own attributes and operations to those inherited from its parent (super or base) class
  - Enables code reuse among related classes
- Polymorphism
  - An object (instance) of derived class “cat” is also an instance of its parent class “animal”
  - An operation of a parent class can be redefined in a derived class, possibly with a different behavior

Inheritance



A subclass may inherit the structure and behavior of its superclass.

Image source: Object-Oriented Analysis and Design with Applications by Grady Booch et al.

Polymorphism

One idea (entity or operation), different forms







Google Images

What are the Classes in your code?

- *Jerry's Cheese Army vs. Tom's Traps*

  Jerry invades **Tom’s** kitchen with **cheese clones** that **multiply** over time.

  Tom sets **traps** to **capture** them.

  Some cheese clones are **trap-resistant**, meaning they can **survive** and keep duplicating.

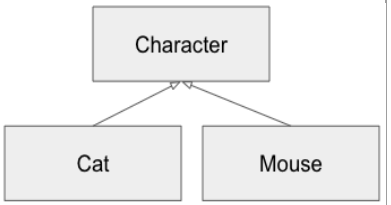
Entity-Relational Model

Entity can be any object or a class that represents a subject in the problem or solution space

- e.g. student, employee

- Attributes are properties or characteristics of an entity, e.g. student-name, employee-id, address etc.
- Relationships help to model how entities interact with other entities:
  - e.g. employee “is-a” person, ”works-for” a department
- Cardinality of relationships: one-one, one-many, many-one, many-many

Types of Relationships between Entities

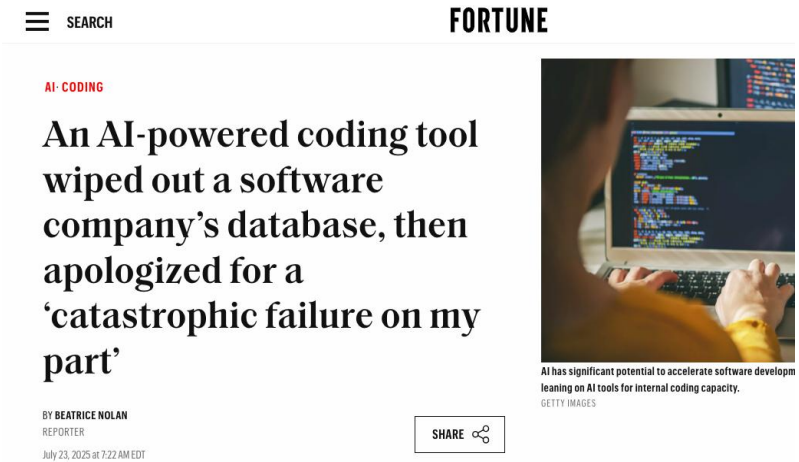
Association: *Tom uses Trap*	***Aggregation:*** *Jerry's house has furniture*	***Composition:*** *Jerry's house has a door (door does not have independent existence)*	***Generalization & Specialization:*** *Tom is a Character* *Jerry is a Character*
			

Google Images

Types of Relationships between Entities

Association: *Two entities are related, but one does not own another*	***Aggregation:*** *"has-a" relationship (ownership), but independent existence*	***Composition:*** *"has-a" relationship, Dependent existence*	***Generalization & Specialization:*** *"is-a" relationship*
Teacher – Student Car – Driver Customer – Order Train – Station Doctor – patient Student – course ...	Library – book Team – player Shop – groceries Folder – file ...	Book – pages Clock – hands Chair – legs Cup – handle Bulb – filament Fan – motor Car – wheel ...	Car – Vehicle Apple – Fruit Mosquito – insect Cheese – food Doll – toy ...

Booting Up the Course



[AI-powered coding tool wiped out a software company's database in 'catastrophic failure' | Fortune]



Google Images